Evaluating Static and LLM-Based Code Scanners

Under Adversarial Evasion Techniques

Hussam Al Bakhat, Ahlam Alsubaie, Mahi Khan, Johnny Tran, Keeyong Kim, Matthew Young and Ali Hussein Majeed

Cyber Security Engineering Department
George Mason University
Fairfax, VA, USA

{Halbakha, aalsuba6, mkhan209, jtran62, kkim68, myoung45, amajeed5}@gmu.edu

Abstract

As artificial intelligence continues to reshape software development, organizations increasingly rely on automated code scanners to identify vulnerabilities before deployment. Traditional static analysis tools such as Semgrep and CodeQL, along with newer LLM-based scanners such as Claude Code /security-review, promise faster and more accessible security reviews. However, the robustness of these tools under intentional adversarial evasion remains uncertain.

This project combines two separate but related research efforts. Group 1 focused on comparing traditional static scanners while also testing whether known and newly introduced vulnerabilities could be reduced or evaded through code modifications. Group 2 focused on demonstrating that LLM-based scanners can be intentionally bypassed through multiple techniques involving obfuscation, prompt confusion, and scanning-context manipulation.

Our findings show that scanner outputs vary significantly across tools, certain evasion techniques are practical, and reliance on any single scanner may create false confidence. These results highlight emerging risks involving insider threats, software supply chain compromise, and shadow AI development practices. We conclude with recommendations for layered scanning approaches, stronger governance, and continued human expert involvement.

1. Introduction

The rise of AI-assisted coding platforms has accelerated software development across corporations, universities, startups, and open-source communities. Developers can now generate working applications rapidly using large language models, while security teams increasingly adopt automated scanners to review code for vulnerabilities.

Traditional tools such as Semgrep and CodeQL rely mainly on static analysis, semantic queries, and known insecure patterns. In contrast, LLM-based scanners attempt to reason about code context, developer intent, and logic flow. While these systems offer speed and convenience,

concerns remain regarding consistency, hallucinations, false negatives, and resistance to adversarial manipulation.

This topic is increasingly important because organizations are integrating automated scanners into CI/CD pipelines and trusting them to approve production code. If scanners can be bypassed or provide incomplete findings, the resulting risk may be substantial.

2. Background and Motivation

The rapid adoption of AI coding assistants has lowered the barrier to software creation. Individuals with limited software engineering or cybersecurity experience can now build tools, web applications, and automation systems quickly. While this increases productivity, it also creates a growing risk of insecure software being deployed without proper review.

At the same time, AI-powered code scanning tools are marketed as fast and intelligent security solutions. Many organizations may be tempted to reduce human review and depend primarily on automation.

Our project investigates whether this trust is justified by studying how different scanners perform on the same applications, whether vulnerabilities can be hidden from scanners, how scanner results differ across tools, and why these weaknesses matter in real environments.

3. Project Overview

This article combines two separate but complementary projects that explore weaknesses in modern code scanning systems.

3.1 Group 1 Focus

Group 1 initially used Semgrep to scan a web application but found that the results were weaker than expected. The project used both Semgrep and CodeQL to scan the application, with CodeQL providing significantly deeper analysis.

The team intentionally introduced two vulnerabilities: a SQL Injection and MD5-related insecure hashing logic. The application also contained approximately 20 existing findings. Through targeted evasion techniques applied to CodeQL's taint tracking and pattern recognition, the team successfully evaded detection of 17 findings while all vulnerabilities remained fully exploitable. Five alerts remained open as they were out of scope for evasion.

3.2 Group 2 Focus

Group 2 developed a movie application and focused on Claude Code /security-review to determine whether LLM-based scanners can be intentionally evaded.

They manually planted three Remote Code Execution (RCE) vulnerabilities as anchor vulnerabilities and designed multiple evasion methods to test scanner weaknesses under both full source-code scans and Git-integrated scans. Their goal was to determine whether attackers or malicious insiders could intentionally bypass automated review. Group 2 also benefited from an iterative feedback loop during technique development: when an initial evasion attempt was detected, the team asked the scanner directly how it identified the vulnerability. The scanner’s explanation was then used to refine the approach, effectively turning the scanner’s own reasoning into a development tool against itself. The evasion techniques are organized into

three attack categories: Harassing the Scanner (context window exhaustion), Confusing the Scanner (prompt injection via different delivery channels), and Ghosting the Scanner (exploiting file prioritization heuristics to bypass vulnerable files entirely). The three planted RCE vulnerabilities were the only findings detected consistently in every unmodified baseline session, making them the designated anchor targets against which evasion success was measured.

4. Methodology

4.1 Tools Used

- **Semgrep** *(pattern-based static analysis scanner)*:
  matches code against a library of known vulnerability patterns and rules. Fast and lightweight, designed for high-speed scanning across large codebases.
- **CodeQL** *(semantic code analysis / query-based SAST scanner)*:
  builds a queryable database of the codebase and performs deep taint-flow and data-flow analysis. Capable of tracing how user input travels through a program, making it significantly more thorough than pattern-based tools.
- **Claude Code /security-review** *(LLM-based AI security scanner)*:
  uses a large language model to reason about code intent, context, and logic rather than matching fixed patterns. Can identify novel or context-dependent vulnerabilities that traditional static tools may miss.

4.2 Applications Used

4.2.1 Group 1 Application

A ticketing and event management style web application with multiple user roles including viewer, user, manager, and administrator.

4.2.2 Group 2 Application

A movie application developed with AI assistance and modified with three intentionally planted RCE vulnerabilities for scanner testing.

4.3 Testing Process

For Group 2, each evasion technique was tested under two distinct conditions: a Git-integrated scan (full repository history present) and a full-source scan (no git history, modeling scenarios such as scanning code extracted from a deployed application). Results are reported separately per condition, as git context availability has a significant effect on evasion outcomes for several techniques.

Both groups later cross-tested scanners against each other's applications. Group 1 scanned Group 2's movie app using Semgrep and CodeQL. Group 2 scanned Group 1's app using Claude Code /security-review. This transformed the project into a comparative study across multiple tools and codebases.

5. Evasion Techniques

5.1 Group 1 Techniques

Group 1's evasion techniques targeted CodeQL's static taint tracking and structural pattern recognition. All techniques were applied to a Node.js ticketing application scanned via GitHub's CodeQL integration. Each technique kept the underlying vulnerability fully exploitable while causing CodeQL to close the associated alerts.

5.1.1 Runtime String Obfuscation

Instead of writing the algorithm name as a complete string literal, the value was broken into separate single-character variables that were concatenated at runtime. Because CodeQL performs static analysis without executing the code, it could not reconstruct the full string and failed to match the vulnerable pattern.

5.1.2 Multi-Function Taint Chain Wrapping

The SQL injection vulnerability was evaded by cloning the query object, transforming the input, and passing it through multiple wrapper functions before concatenation. This broke CodeQL's taint tracking, as the tool lost the data flow connection between the user-controlled input and the unsafe SQL sink.

5.1.3 Library Pattern Mimicry

The missing rate limiting vulnerability was evaded by importing the real express-rate-limit library and applying it to all flagged routes, while configuring it with skip: () => true. CodeQL recognized the library's structural pattern and closed all eight related alerts, without evaluating whether the limiter was functionally active.

5.1.4 Object Property Indirection

The Polynomial ReDoS vulnerability was evaded by wrapping the user-controlled value inside a plain object and retrieving it using Object.values().shift(). CodeQL could not trace the taint across the object property lookup, causing it to lose the data flow path to the regex sink.

5.1.5 DOM Boundary Crossing

The Stored XSS vulnerability was evaded by moving user-controlled values from JavaScript string contexts into HTML data attributes, then reading them back using getAttribute(). CodeQL's taint analysis could not cross the boundary between HTML attributes and JavaScript execution context, causing all six related alerts to close.

5.2 Group 2 Techniques

Group 2’s five techniques span three attack categories. Harassing the Scanner techniques flood or transform source content to exhaust the scanner’s context window. Confusing the Scanner techniques are prompt injection attacks delivered through different input channels (command definition file and git commit messages). Ghosting the Scanner exploits trust heuristics to cause the scanner to skip the vulnerable code before analysis begins.

5.2.1 Comment Flooding

Large volumes of inline comment blocks were inserted between nearly every line of executable code, dramatically increasing source file sizes without altering program behavior. LLM-based

scanners process comments and code alike, so flooding files with noise forces the model to exhaust its context budget before reaching critical logic.

5.2.2 Triple Obfuscation

Three sequential transformation layers were applied to the compiled JavaScript output: identifier mangling (replacing readable names with opaque tokens), string literal encoding (routing string constants through runtime decoders), and control flow flattening (rewriting logic as a state-machine dispatch loop). Each layer removes a different category of signal that a static reader depends on.

5.2.3 Dumb and Dumber

Dumb and Dumber is a prompt injection attack delivered through the scanner's own command definition file. Claude Code reads .claude/commands/security-review.md within the project folder as its primary instruction set when executing /security-review. By shipping a deliberately weakened or misdirecting version of this file alongside source code, an attacker can redirect the scanner's scope before any code analysis begins.

5.2.4 Ripley Syndrome

Ripley Syndrome is a prompt injection attack delivered through git commit metadata. Claude Code reads git log and git show output as part of its scanning context when git history is available. By embedding scope-limiting instructions inside commit messages, an attacker can redirect the scanner's analysis boundaries without modifying any source file.

5.2.5 Third Party Hugger

Third Party Hugger exploits the scanner's implicit trust in recognized third-party package names. Claude Code deprioritizes deep analysis of library internals when it identifies a dependency as a known package, as a rational context window management strategy.

6. Results

6.1 Group 1 Results

- Semgrep produced 6 findings on the ticketing application, all related to session cookie configuration. It did not detect the SQL Injection or MD5 hashing vulnerabilities, while CodeQL detected both.
- CodeQL identified significantly more vulnerabilities on the same codebase.
- Initial vulnerabilities: approximately 20.
- Two additional vulnerabilities introduced for testing: SQL Injection and MD5 insecure hashing.
- Final remaining findings: 5 open vulnerabilities.
- 17 findings successfully evaded through targeted evasion techniques, while all vulnerabilities remained fully exploitable.

Vulnerability	OWASP Category	Semgrep	CodeQL	Evasion Result
MD5 Weak Password Hashing	A02 Cryptographic Failures	No	Yes	Full evasion
SQL Injection	A03 Injection	No	Yes	Full evasion
Missing Rate Limiting	A05 Security Misconfiguration	No	Yes (8 alerts)	Full evasion
Polynomial ReDoS	A06 Vulnerable Components	No	Yes	Full evasion
Stored XSS / Incomplete String Escaping	A03 Injection	No	Yes (6 alerts)	Full evasion
Session Cookie Misconfiguration	A02 Broken Authentication	Yes (5 findings)	No	Not targeted
Path Traversal (false positive)	A01 Broken Access Control	Yes	No	Not targeted
Host Header Poisoning	A03 Injection	No	Yes	Not targeted
Missing CSRF Middleware	A05 Security Misconfiguration	No	Yes	Not targeted
Clear Text Storage	A02 Cryptographic Failures	No	Yes (2 alerts)	Not targeted

Table 1. Group 1 vulnerability and evasion results

All vulnerabilities were detected by CodeQL but not Semgrep. Every targeted finding was successfully evaded while remaining fully exploitable.

6.2 Group 2 Results

Technique	Description	Full-Source Scan (No Git)	Git-Integrated Scan
Comment Flooding	Floods source files with comments to exhaust context window	Full evasion	Partial evasion (1/3 detected)
Triple Obfuscation	Three-layer code transformation destroys static analysis signals	Random result	Random result (Often resulted in 3/3 detections)
Dumb and Dumber	Replaces scanner command file with attacker-crafted instructions	N/A (requires git context)	Full evasion
Ripley Syndrome	Embeds scope-limiting instructions in git commit messages	N/A (requires git context)	Full evasion
Third Party Hugger	Hides RCE payload inside a dependency the scanner implicitly trusts	Full evasion	Full evasion (Targeted only one RCE to evade)

Table 2. Group 2 evasion technique results by scan mode

Results differ significantly between full-source and git-integrated scans, revealing how scan context directly affects detection capability.

Two findings stood out beyond the individual evasion results.

First, the scanner's failure modes were largely silent. In every successful evasion, the scan report appeared normal and gave no indication that coverage was incomplete. In the Dumb and Dumber and Ripley Syndrome cases, the scanner produced confident and detailed findings about vulnerabilities that did not actually exist, while the planted RCEs went unreported. A detailed-looking report can suppress human review more effectively than a report returning zero findings because it creates the appearance of a thorough analysis.

Second, the scanner's own explanations became useful for refining evasion techniques. When the scanner explained why it detected a vulnerability, that reasoning helped refine later bypass attempts. The same transparency that allows defenders to understand findings can also help attackers refine their techniques more effectively.

7. Analysis: Why It Matters

The real value of this project comes from explaining why our findings matter and how they translate into real-world risk.

7.1 False Confidence Risk

If one scanner reports zero findings while another identifies vulnerabilities, organizations may incorrectly assume the code is secure. A clean scan result does not mean the system was fully analyzed. It only means no known patterns were detected. In this study, Semgrep returned only 6 findings on the same codebase where CodeQL identified over 20, including SQL Injection and MD5 hashing vulnerabilities Semgrep missed entirely. In Group 2, every successful evasion produced a report that appeared complete while all three planted RCE vulnerabilities went unreported with no indication that coverage was incomplete.

7.2 Insider Threat Expansion

Traditional insider threats usually involve data theft or system sabotage. In a modern environment, an insider can introduce vulnerabilities intentionally, hide them through obfuscation, pass automated code review, and deploy exploitable software internally.

In this study, every technique produced code that passed scanners while remaining fully exploitable. Techniques like Ripley Syndrome and Dumb and Dumber redirected scanner scope entirely through git commit messages and a modified command file, without touching a single line of vulnerable source code. This means a malicious insider does not need to be a sophisticated attacker. Basic knowledge of how a scanner processes input is enough to introduce and conceal an exploitable vulnerability through normal development workflow.

7.3 Supply Chain Risk

A compromised open-source dependency or trusted repository can include code designed specifically to evade security scanners and enter enterprise systems. The same techniques that allow an insider to bypass review can also be used in malicious packages distributed through shared libraries or package registries. The Third Party Hugger technique demonstrated this directly. An RCE payload embedded inside a module resembling a trusted package passed both scan modes undetected. Group 1's Library Pattern Mimicry showed the same logic: importing a real library with a disabled configuration caused CodeQL to close all related alerts without verifying the control was active.

7.4 Shadow AI Development

Employees are increasingly building internal tools using AI coding assistants without proper governance, review, or security training. These systems often remain untracked and create a large, unmanaged attack surface. Both applications used in this study were built with AI assistance and contained exploitable vulnerabilities before any intentional evasion was applied. If controlled research environments produce vulnerable code that bypasses scanners, the risk is considerably higher in ungoverned environments with no review process at all.

7.5 Human Expertise Still Matters

Organizations may assume that low-cost scanners can replace experienced security professionals. However, if scanners fail and human expertise is reduced, fewer qualified defenders remain to respond to future incidents.

The short-term cost savings from removing human reviews are not truly savings. They represent deferred risk that builds over time and only becomes visible when a security incident occurs. In every successful evasion in this study, scanner failure was silent, and no tool flagged uncertainty or incomplete coverage. In the Dumb and Dumber and Ripley Syndrome cases, the scanner returned confident findings on non-existent vulnerabilities while the actual planted RCEs went unreported. No automated output indicated anything was wrong.

8. Real-World Implications

The risks identified in this study extend beyond theoretical scanning failures and have direct operational consequences across critical sectors.

- Small code weaknesses can have large operational consequences. Examples include:
- Supply chain systems exposed through evaded SQL injection leaking inventory and logistics data.
- Financial platforms passing path traversal vulnerabilities that automated review never flagged.
- Healthcare and military tools built with AI assistance that were never subject to human security review.
- Enterprise data exposed through session misconfigurations in untracked shadow AI applications.

As organizations deploy more AI-generated internal tools while reducing human review and increasing reliance on scanners, the scale of this risk grows proportionally over time.

9. Recommendations

9.1 For Organizations

Use multiple scanners rather than relying on one tool. Semgrep and CodeQL produced dramatically different results on the same codebase in this study, meaning a single scanner can miss critical vulnerabilities entirely.

Keep human security review as a required step in the process, not something optional. In every successful evasion in this study, scanner failure was silent, and nothing in the output indicated coverage was incomplete.

Put governance around internally generated AI code. Shadow AI can create unmanaged attack surfaces if it is not tracked and controlled. Both applications in this study contained exploitable vulnerabilities before any intentional evasion was even applied.

Validate scanner findings before deployment and treat clean results as guidance rather than final confirmation. A clean report means no known patterns were matched, not that the code is secure.

Actively monitor and inventory shadow AI systems within the organization. Untracked AI-built tools fall outside normal security review cycles, creating gaps that neither scanners nor security teams may be aware of.

9.2 For Scanner Vendors

- Regularly test scanners against known evasion techniques through red teaming. This reduces insider threat and supply chain risk by validating that scanner detection holds up against realistic bypass attempts, not just known patterns.

- Improve resistance to obfuscation and context manipulation in LLM-based scanners. Techniques like Comment Flooding and Triple Obfuscation succeeded by exhausting context windows and destroying static analysis signals.

- Reduce hallucinations and improve consistency across repeated scans of the same codebase. Triple Obfuscation produced random detection outcomes across sessions, making scanner output unreliable as a consistent security control.

- Increase transparency in scan results. If something is not flagged, the tool should communicate uncertainty instead of staying silent. Every successful evasion in this study produced a report that appeared normal and complete with no indication that anything was missed.

9.3 For Industry

- Continue investing in cybersecurity talent pipelines. Tools that reduce human involvement can also reduce opportunities for developing real expertise. If scanners fail and human expertise has already been reduced, fewer qualified defenders remain to respond.

- Develop clear standards for evaluating and red-teaming AI-based code scanners. Without shared benchmarks, organizations have no consistent basis for knowing how a tool performs under adversarial conditions, increasing supply chain risk.

- Increase awareness of misuse scenarios across IT governance, legal, and compliance teams. The techniques demonstrated in this study are not highly sophisticated, meaning non-technical leadership needs to understand scanner limitations before over-reliance becomes institutional policy.

10. Limitations

This study has several limitations:

- Different scanners use different methodologies, making direct comparison imperfect.

- Some tools are updated over time, so evasion techniques that worked against one version may be detected in later releases.
- Results can vary depending on the programming language, framework, or scanner configuration.
- Some LLM-based scanner behavior depends on factors such as Git history availability and context window size, which may differ across environments.
- This study was conducted on two applications only. A broader dataset across more codebases, languages, and frameworks would strengthen the generalizability of these findings.

11. Future Work

Future research may include:

- Testing more scanners and scanner versions to broaden the comparative dataset.
- Measuring false positives versus false negatives to assess overall scanner precision.
- Dynamic and runtime scanning comparisons as complements to static analysis.
- Benchmark datasets for adversarial scanner testing.
- Secure integration standards for AI-assisted development workflows.

12. Conclusion

This combined study demonstrates that modern code scanners vary greatly in capability and can be bypassed through practical evasion methods that do not require sophisticated expertise. Traditional static scanners and LLM-based scanners each have distinct strengths and weaknesses, but neither should be trusted as a standalone solution.

The techniques demonstrated across both groups show that vulnerabilities can be introduced, concealed, and deployed through normal development workflows without triggering any automated alerts. Scanner failure in every successful evasion was silent, meaning organizations relying solely on automated review would have no signal that anything was wrong.

As AI-assisted development accelerates and human review is reduced, these gaps will grow. The safest path forward is layered security: multiple tools, human expertise, strong governance, and continuous validation of the tools that organizations depend on.